

Preprocessing with recipes

2026-04-17

Introduction

The **recipes** package in **tidymodels** defines preprocessing pipelines as a sequence of named steps — impute missing values, normalise numeric predictors, encode categoricals, apply PCA — that learn their parameters from training data and apply them consistently at prediction time. The principal benefit is leak-free preprocessing: imputation means, scaling parameters, and PCA loadings are estimated only on the training set, never on test or validation folds. Applying preprocessing globally before splitting (a frequent mistake) leaks information from test into training and produces optimistically biased performance estimates.

Prerequisites

A working understanding of **tidymodels**, train-test splits, cross-validation, and the concept of data leakage.

Theory

A recipe has two phases:

1. **Define**: chain **recipe()** with **step_*()** functions in pipeline order.
2. **Prep / bake**: **prep()** learns parameters from training data; **bake()** applies them to any new data.

All statistics (centring means, scaling SDs, imputation values, PCA loadings) come from the data passed to **prep()**. When the recipe is then applied to test data via **bake()** or via a **workflow()**, the same parameters are used — preserving the principle that test data must not influence preprocessing.

Steps are executed in declared order; ordering matters substantively (impute before scale, not after; encode categoricals before any operation that requires numeric inputs).

Assumptions

Training data is representative enough to estimate transform parameters reliably; step ordering is correct for the chosen transformations.

R Implementation

```
library(recipes); library(dplyr)

set.seed(2026)
n <- 200
df <- tibble(
  num1 = rnorm(n),
  num2 = rnorm(n),
  cat  = sample(c("a", "b", "c"), n, replace = TRUE),
  y    = rnorm(n)
)
df$num1[sample(n, 10)] <- NA           # introduce missingness

rec <- recipe(y ~ ., data = df) %>%
```

```

step_impute_mean(num1) %>%
step_normalize(all_numeric_predictors()) %>%
step_dummy(all_nominal_predictors())

prepped <- prep(rec, training = df[1:150, ])
test_baked <- bake(prepped, new_data = df[151:200, ])
head(test_baked)

```

Output & Results

`prep()` returns a fitted recipe with all learnt parameters stored. `bake()` applies the prepared recipe to new data, returning a transformed tibble with imputed, normalised, and one-hot-encoded predictors. The transformations on the held-out 50 rows use parameters learned only from the first 150.

Interpretation

A reporting sentence: “The recipe imputed missing values in `num1` with the training-set mean (-0.02), centred and scaled all numeric predictors, and one-hot encoded the `cat` factor with three levels; all parameters were learned on the 150-row training set and applied identically to the 50-row test set, avoiding the data leakage that occurs when preprocessing is applied globally before splitting.” Always describe the preprocessing steps in the methods section.

Practical Tips

- Place imputation steps before scaling; you cannot meaningfully standardise a column that contains NAs.
- `step_normalize()` centres and scales; `step_range()` rescales to $[0, 1]$ via min-max.
- For tree-based models (random forest, XGBoost), skip normalisation — trees are scale-invariant — but keep imputation and one-hot encoding.
- Use `step_zv()` and `step_nzv()` to remove zero-variance and near-zero-variance predictors before modelling; they contribute no information and can destabilise some algorithms.
- Pair a recipe with a model specification via `workflow()` so preprocessing and prediction happen as one pipeline; this prevents accidental leakage at predict time.
- For PCA dimensionality reduction, `step_pca()` learns loadings on training data; the same loadings are applied to test data, preserving train-test independence.

R Packages Used

`recipes` for the core preprocessing framework; `workflows` for pairing recipes with models; `embed` for embedding steps (target encoding, UMAP); `themis` for class-imbalance steps (SMOTE, ROSE); `textrecipes` for text-specific preprocessing; `tidymodels` as the umbrella package that bundles all of these.

For Reviewers

What to look for in a paper using this method.

- Common misapplications.
- Diagnostics that should be reported but often aren’t.
- Red flags in tables and figures.
- What to verify.
- What an adequate Methods paragraph must contain.

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+

- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting
- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro
- tidymodels pipelines
- FDR, knockoffs, replication crisis